

Arquitectura de Microservicios y Mapa de Dominios

Santo Pegasus Soluciones

Documento: Arquitectura de Microservicios y Mapa de Dominios

Versión: 1.0.0

Fecha de Emisión: Junio 2026

Departamento: Ingeniería de Software / Chapter de Back-end

Clasificación: Interno — Uso Técnico Restringido

TABLA DE CONTENIDOS

1. Introducción y Visión General de la Arquitectura
2. Diagrama Textual de la Arquitectura General
3. Catálogo Completo de Microservicios
4. Mapa de Dependencias entre Servicios
5. Patrones de Comunicación
6. Estrategia de Bases de Datos
7. API Gateway
8. Infraestructura en AWS
9. Observabilidad Distribuida
10. Estrategia de Versionado de APIs
11. Seguridad entre Servicios
12. Mapa de Squads y Ownership
13. Roadmap Técnico
14. Architecture Decision Records (ADRs)
15. Disposiciones Finales y Proceso de Actualización

SECCIÓN 1 — INTRODUCCIÓN Y VISIÓN GENERAL DE LA ARQUITECTURA

1.1 Propósito del Documento

Este documento constituye el mapa arquitectónico oficial de Santo Pegasus Soluciones. Su propósito es servir como referencia canónica para todos los ingenieros, Tech Leads, Product Managers y stakeholders técnicos que necesiten comprender cómo están organizados, interconectados y desplegados los sistemas que componen el portafolio de productos de la empresa.

Este artefacto debe leerse en conjunto con la Guía Oficial de Ingeniería Back-end (v2.4.0), que establece los estándares de codificación, patrones de diseño y prácticas de seguridad obligatorias para todos los servicios aquí descritos. Ambos documentos forman el núcleo de la base de conocimiento técnico de Santo Pegasus y son fuentes de verdad para la toma de decisiones arquitectónicas.

1.2 Contexto Empresarial

Santo Pegasus Soluciones es una empresa de tecnología especializada en el desarrollo de productos digitales para el sector de salud y servicios profesionales. El producto principal de la compañía es Agendio, una plataforma SaaS de agendamiento de consultas médicas que conecta pacientes, médicos y clínicas a través de una experiencia digital integrada.

La plataforma Agendio opera en modelo multi-tenant, atendiendo a redes de clínicas, hospitales de pequeño y mediano porte, y consultorios independientes en todo Brasil. La criticidad del dominio de salud exige que la arquitectura priorice disponibilidad, seguridad de datos sensibles, conformidad con la LGPD (Lei Geral de Proteção de Dados) y auditabilidad completa de todas las operaciones.

1.3 Por Qué Microservicios

La adopción de una arquitectura de microservicios en Santo Pegasus no fue una decisión tomada de forma prematura. La empresa comenzó con un monolito funcional, y la migración para microservicios fue guiada por necesidades reales e inmediatas de escala, autonomía de equipos y velocidad de entrega, en consonancia con los principios fundamentales documentados en la Guía de Ingeniería Back-end.

Las motivaciones concretas que justificaron la transición son:

- **Escalabilidad Independiente:** El servicio de agendamiento (`agendio-scheduling-service`) presenta picos de carga en horarios matutinos (entre 07h00 y 09h00) que no impactan otros dominios. En arquitectura monolítica, escalar ese módulo implicaría escalar toda la aplicación, desperdiciando recursos computacionales y aumentando costos operativos.
- **Autonomía de Squads:** Con equipos organizados por dominio de negocio, la arquitectura de microservicios permite que cada squad tenga ownership completo de su ciclo de vida de software — desde el código hasta el deploy en producción — sin coordinación burocrática con otros equipos para releases.

- **Resiliencia e Isolamento de Fallas:** Un defecto crítico en el ``payment-service`` no debe causar la indisponibilidad del agendamiento de consultas. El aislamiento de procesos garantiza que las fallas se contengan dentro del dominio afectado.
- **Flexibilidad Tecnológica:** Diferentes dominios tienen diferentes requisitos técnicos. El ``medical-records-service`` se beneficia de MongoDB para el almacenamiento de documentos clínicos flexibles y semi-estructurados, mientras que el ``auth-service`` requiere la consistencia fuerte de PostgreSQL para la gestión de credenciales.
- **Velocidad de Entrega:** La independencia de deploys permite que múltiples squads liberen nuevas funcionalidades en el mismo día sin coordinación de releases ni ventanas de mantenimiento compartidas.

1.4 Principios Arquitectónicos que Guían las Decisiones

Todas las decisiones arquitectónicas en Santo Pegasus están ancladas en los siguientes principios, que deben ser internalizados por todo el equipo de ingeniería:

#	Principio	Descripción
P-01	High Cohesion, Low Coupling	Cada microservicio debe encapsular un dominio de negocio bien definido. Dependencias entre servicios deben ser minimizadas y, cuando existentes, preferiblemente asíncronas.
P-02	Database per Service	Cada microservicio posee y controla exclusivamente su propia base de datos. El acceso directo a la base de datos de otro servicio está terminantemente prohibido.
P-03	API First	Contratos de API (OpenAPI/Swagger) son definidos antes de la implementación. Esto garantiza que los consumidores puedan desarrollar en paralelo usando mocks.
P-04	Security by Default	Todas las comunicaciones entre servicios son autenticadas. Ningún endpoint interno es accesible sin validación de identidad. mTLS es obligatorio para comunicación interna.
P-05	Observability by Design	Logs estructurados, métricas y rastreo distribuido no son añadidos post-hoc; son parte del scaffolding inicial de cada nuevo servicio.
P-06	Fail Fast, Recover Gracefully	Los servicios implementan circuit breakers, timeouts y políticas de retry con backoff exponencial para garantizar resiliencia ante fallas de dependencias.

P-07	Compliance First	Decisiones sobre almacenamiento, transmisión y procesamiento de datos sensibles de salud son guiadas por la LGPD y buenas prácticas de seguridad antes de cualquier requisito funcional.
P-08	Infrastructure as Code	Toda la infraestructura AWS es definida como código (Terraform/AWS CDK). Ningún recurso de producción es creado manualmente via Console.
P-09	Evolutionary Architecture	La arquitectura acepta que cambiará. Decisiones son tomadas con horizontes de tiempo claros y revisadas periódicamente en los foros de Architecture Review.
P-10	SOLID in Every Service	Los principios SOLID de orientación a objetos son aplicados tanto a nivel de clase como a nivel de servicio. Cada microservicio tiene una única responsabilidad de dominio.

1.5 Ecosistema Tecnológico Principal

El ecosistema tecnológico de Santo Pegasus está standardizado para garantizar consistencia operacional y reducir la carga cognitiva de los equipos:

- **Lenguaje & Runtime:** Java 17+ (LTS), con adopción gradual de Java 21 (Virtual Threads)
- **Framework Principal:** Spring Boot 3+, Spring Security, Spring Cloud
- **Contenedorización:** Docker (imágenes inmutables), orquestados en AWS ECS Fargate
- **Bases de Datos:** PostgreSQL (relacional), MongoDB/AWS DocumentDB (documentos), Redis/AWS ElastiCache (caché)
- **Mensajería:** AWS SQS (colas y eventos entre dominios)
- **Autenticación:** JWT + OAuth 2.0 / OpenID Connect, gestionado por `auth-service`
- **Observabilidad:** SLF4J + Logback, Spring Boot Actuator, Micrometer, Prometheus, Datadog
- **CI/CD:** GitHub Actions + Pipelines con Docker, Flyway/Liquibase para migrations
- **Secrets Management:** AWS Secrets Manager + Spring Cloud Config

SECCIÓN 2 — DIAGRAMA TEXTUAL DE LA ARQUITECTURA GENERAL

2.1 Visión de Alto Nivel

El diagrama a continuación representa la topología completa de la arquitectura de microservicios de Santo Pegasus, mostrando el flujo de tráfico desde los clientes externos hasta los servicios internos, bases de datos y sistemas de mensajería.

...

CLIENTES EXTERNOS

[Web App React] [Mobile App iOS/Android] [Clínicas API Partners]

▼ HTTPS / TLS 1.3

AWS API GATEWAY (Kong / AWS API GW)

- Routing
- Rate Limiting
- Auth Token Validation
- SSL Termination
- CORS
- Request Logging



auth-service :8081
user-service :8082
agendioscheduling-service :8083
payment-service :8085
ai-assistant-service :8087

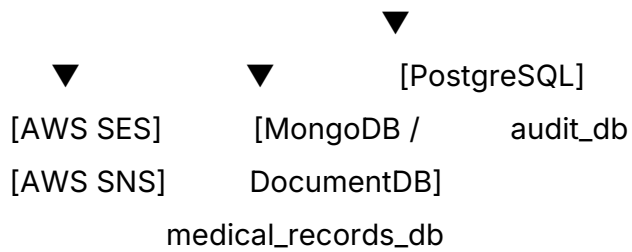


[PostgreSQL] auth_db
[PostgreSQL] users_db
[PostgreSQL] payments_db
[Pinecone DB] (Vector Store)



agendionotif-service
medicalrecords-service
audit-service :8088

:8086



CAPA DE MENSAJERÍA ASÍNCRONA

AWS SQS QUEUES

```
appointment-created.fifo → [agendio-notification-service]
appointment-cancelled.fifo → [agendio-notification-service]
                             → [audit-service]
payment-confirmed.fifo → [agendio-scheduling-service]
                        → [audit-service]
user-created.fifo      → [agendio-notification-service]
                        → [audit-service]
audit-events.fifo      → [audit-service]
```

INFRAESTRUCTURA COMPARTIDA

AWS	AWS	AWS Secrets
ElastiCache	CloudWatch	Manager
(Redis)	+ Datadog	+ Spring Cloud
Caché Global	Observability	Config

...

2.2 Flujo de una Requisición Típica (Agendamiento de Consulta)

...

Cliente (App Mobile)

POST /v1/appointments [Bearer JWT]



API Gateway

- Valida JWT via auth-service (cache Redis, TTL 5min)
- Aplica rate limit (100 req/min por tenant)
- Routea para agendio-scheduling-service



agendio-scheduling-service

- Valida disponibilidad del médico (consulta propia DB)
- Verifica perfil del paciente via user-service (REST síncrono)
- Crea registro de consulta en scheduling_db (PostgreSQL)
- Publica evento "appointment-created" en SQS
- Retorna 201 Created con payload de la consulta



[Async] agendio-notification-service consume SQS

- Envía email de confirmación via AWS SES
- Envía SMS via AWS SNS



[Async] audit-service consume SQS

- Registra acción en audit_db con Trace ID completo

...

SECCIÓN 3 — CATÁLOGO COMPLETO DE MICROSERVICIOS

3.1 Resumen del Catálogo

Servicio	Puerto	Dominio	Base de Datos	Squad Owner
`auth-service`	8081	Identidad & Seguridad	PostgreSQL	Squad Hermes
`user-service`	8082	Usuarios & Perfiles	PostgreSQL	Squad Hermes
`agendio-scheduling-service`	8083	Agendamiento (Core)	PostgreSQL	Squad Agendio Core
`agendio-notification-service`	8084	Notificaciones	— (stateless)	Squad Agendio Core
`payment-service`	8085	Pagos & Facturación	PostgreSQL	Squad Pagamentos
`medical-records-service`	8086	Historial Clínico	MongoDB	Squad Clínico

`ai-assistant-service`	8087	Asistencia IA (RAG)	Pinecone (Vector)	Squad IA
`audit-service`	8088	Auditoría & Compliance	PostgreSQL	Squad Governance

3.2 `auth-service` — Autenticación y Autorización

Repositorio Git: `git@github.com:santopegasus/auth-service.git`

Squad Owner: Squad Hermes

Tech Lead: Isabella Carvalho

Puerto: 8081

Responsabilidad

El `auth-service` es la raíz de confianza del ecosistema de Santo Pegasus. Es responsable de la emisión, validación y revocación de tokens JWT, de la gestión del ciclo de vida de credenciales de usuarios, y de la integración con el flujo OAuth 2.0 / OpenID Connect para clientes de terceros. Ningún otro servicio valida credenciales directamente; toda autenticación pasa obligatoriamente por este servicio.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Security 6
- **Autenticación:** JWT (RS256), OAuth 2.0, OpenID Connect (OIDC)
- **Base de Datos:** PostgreSQL 15 (`auth\_db`) via AWS RDS
- **Caché de Tokens:** AWS ElastiCache (Redis) — TTL configurable por tipo de token
- **Secrets:** AWS Secrets Manager para llaves RSA privadas

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/auth/login`	Autenticación con email/password, retorna JWT + Refresh Token
`POST`	`/v1/auth/refresh`	Renovación de token via Refresh Token
`POST`	`/v1/auth/logout`	Revocación de token (blacklist en Redis)
`POST`	`/v1/auth/oauth2/token`	Flujo OAuth 2.0 Client Credentials
`GET`	`/v1/auth/validate`	Validación de token (usado por API Gateway)
`POST`	`/v1/auth/password/reset`	Solicitud de reset de contraseña

`GET`	`/.well-known/jwks.json`	JWKS endpoint para validación pública de tokens
-------	--------------------------	---

Dependencias con Otros Servicios

- **`user-service` (REST síncrono):** Para validar existencia y status del usuario durante el login
- **`audit-service` (SQS asíncrono):** Publica eventos de ``auth.login.success``, ``auth.login.failure``, ``auth.token.revoked``

Modelo de Datos (auth_db)

...

users_credentials → id, user_id, password_hash, is_active, mfa_secret

refresh_tokens → id, token_hash, user_id, expires_at, is_revoked

roles → id, name, permissions (JSONB)

user_roles → user_id, role_id

oauth_clients → client_id, client_secret_hash, scopes, redirect_uris

...

3.3 `user-service` — Gestión de Perfiles de Usuarios

Repositorio Git: ``git@github.com:santopegasus/user-service.git``

Squad Owner: Squad Hermes

Tech Lead: Isabella Carvalho

Puerto: 8082

Responsabilidad

Gestiona los perfiles de todos los actores del sistema: pacientes, médicos, administradores de clínicas y operadores. Es la fuente de verdad para datos de perfil, como nombre, contacto, especialidades médicas (para el rol Doctor), y configuraciones de cuenta. No almacena credenciales de acceso — esa responsabilidad es exclusiva del ``auth-service``.

Tecnologías

- **Framework:** Spring Boot 3.x
- **Base de Datos:** PostgreSQL 15 (``users_db``) via AWS RDS
- **Caché:** Redis para cachear perfiles frecuentemente accedidos (TTL 10min)
- **Mapeo:** MapStruct para conversión Entidad → DTO
- **Documentación:** springdoc-openapi (Swagger UI)

APIs Expuestas

Método	Endpoint	Descripción
`POST`	<code>`/v1/users`</code>	Creación de nuevo usuario
`GET`	<code>`/v1/users/{id}`</code>	Consulta de perfil por ID

`PUT`	`/v1/users/{id}`	Actualización completa de perfil
`PATCH`	`/v1/users/{id}/status`	Activación / desactivación de cuenta
`GET`	`/v1/doctors`	Listado de médicos con filtros (especialidad, cidade)
`GET`	`/v1/doctors/{id}/availability`	Disponibilidad del médico (proxy para scheduling)
`GET`	`/v1/users/{id}/preferences`	Preferencias de notificación del usuario

Dependencias

- **`audit-service` (SQS asíncrono):** Eventos de ``user.created``, ``user.updated``, ``user.deactivated``
- **`agendio-notification-service` (SQS asíncrono):** Evento ``user.created`` para disparo de email de bienvenida

Modelo de Datos (users_db)

...

users → id (UUID), name, email, phone, cpf_hash, role, status, created_at

doctors → user_id (FK), crm, specialty, bio, consultation_price

patients → user_id (FK), birth_date, health_plan, emergency_contact

addresses → id, user_id (FK), street, city, state, zip_code

preferences → user_id (FK), notify_email, notify_sms, language

...

3.4 `agendio-scheduling-service` — Núcleo de Agendamiento

Repositorio Git: ``git@github.com:santopegasus/agendio-scheduling-service.git``

Squad Owner: Squad Agendio Core

Tech Lead: Rodrigo Mendes

Puerto: 8083

Responsabilidad

Es el servicio más crítico del ecosistema. Gestiona toda la lógica de negocio de agendamiento de consultas médicas: creación, confirmación, cancelación y reagendamiento de consultas; gestión de la agenda de disponibilidad de médicos; control de conflictos de horarios; y reglas de negocio específicas de cada clínica (como políticas de cancelación y límites de consultas por paciente). Ninguna regla de agendamiento reside fuera de este servicio.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Data JPA

- **Base de Datos: PostgreSQL 15 (`scheduling\_db`)** via AWS RDS — transacciones ACID críticas
- **Migrations: Flyway** — versionado de schema en Git junto al código
- **Mensajería: AWS SQS Producer** (publica eventos de ciclo de vida de consultas)
- **Circuit Breaker: Resilience4j** para llamadas a `user-service` y `payment-service`

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/appointments`	Crear nuevo agendamiento
`GET`	`/v1/appointments/{id}`	Consultar agendamiento por ID
`PATCH`	`/v1/appointments/{id}/cancel`	Cancelar consulta
`PATCH`	`/v1/appointments/{id}/reschedule`	Reagendar consulta
`GET`	`/v1/appointments/patient/{patientId}`	Historial de consultas del paciente
`GET`	`/v1/schedules/doctor/{doctorId}`	Agenda de disponibilidad del médico
`POST`	`/v1/schedules/doctor/{doctorId}/slots`	Configurar slots de disponibilidad
`PUT`	`/v1/schedules/doctor/{doctorId}/block`	Bloquear período (vacaciones, ausencias)

Dependencias

- **`user-service` (REST síncrono, Resilience4j):** Validar paciente y médico existentes
- **`payment-service` (REST síncrono):** Verificar status de pago antes de confirmar consulta
- **`audit-service` (SQS):** Eventos de toda alteración de estado de consulta
- **`agendio-notification-service` (SQS):** Publicar eventos para disparo de notificaciones

Eventos SQS Publicados

...

appointment-created.fifo
appointment-confirmed.fifo
appointment-cancelled.fifo
appointment-rescheduled.fifo
appointment-reminder-due.fifo (publicado por scheduler interno, T-24h e T-1h)

...

Modelo de Datos (scheduling_db)

...

appointments → id (UUID), patient_id, doctor_id, clinic_id, status,
scheduled_at, duration_min, type, notes, created_at
schedule_slots → id, doctor_id, day_of_week, start_time, end_time, is_available
blocked_periods → id, doctor_id, start_date, end_date, reason
clinics → id, name, cnpj, address, max_appointments_per_day
...

3.5 `agendio-notification-service` — Notificaciones

Repositorio Git: ``git@github.com:santopegasus/agendio-notification-service.git``

Squad Owner: Squad Agendio Core

Tech Lead: Rodrigo Mendes

Puerto: 8084

Responsabilidad

Servicio stateless responsable de orquestar el envío de todas las notificaciones de la plataforma Agendio: confirmaciones de consulta, recordatorios automáticos, alertas de cancelación y mensajes de bienvenida. Consume eventos de SQS y decide el canal de entrega (email via AWS SES o SMS via AWS SNS) basado en las preferencias del usuario (consultadas en `user-service`). No posee base de datos propia; su estado es integralmente derivado de los eventos recibidos.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Cloud AWS
- **Email:** AWS SES (Simple Email Service) con templates HTML/Thymeleaf
- **SMS:** AWS SNS (Simple Notification Service)
- **Mensajería:** AWS SQS Consumer (escucha múltiples colas)
- **Stateless:** Sin base de datos propia — idempotencia garantizada via SQS deduplication ID

Dependencias

- **`user-service` (REST síncrono):** Consultar preferencias de notificación del usuario
- **AWS SES:** Para envío de emails transaccionales
- **AWS SNS:** Para envío de SMS
- **SQS (Consumer):** Consume eventos de `agendio-scheduling-service` y `user-service`

Templates de Notificación Gestionados

Evento	Canal	Template
`appointment-created`	Email + SMS	Confirmación con código e detalhes
`appointment-reminder-due (T-24h)`	Email + SMS	Recordatorio 24h antes
`appointment-reminder-due (T-1h)`	SMS	Recordatorio 1h antes

`appointment-cancelled`	Email	Confirmación de cancelamiento
`appointment-rescheduled`	Email + SMS	Nuevo horario confirmado
`user-created`	Email	Bienvenida a la plataforma
`password-reset-requested`	Email	Link de reset (expira en 30min)

3.6 `payment-service` — Procesamiento de Pagos

Repositorio Git: ``git@github.com:santopegasus/payment-service.git``

Squad Owner: Squad Pagamentos

Tech Lead: Lucas Andrade

Puerto: 8085

Responsabilidad

Gestiona el ciclo de vida completo de transacciones financieras de la plataforma: procesamiento de pagos via Pix y cartão de crédito/débito, integración con el gateway de pagamento externo (Stripe), gestión de estornos y reembolsos, y generación de recibos. Toda operación financiera es idempotente y completamente auditada. El servicio no almacena datos de cartão en sus propias bases de datos — el tokenizado es responsabilidad del gateway externo (PCI-DSS compliance).

Tecnologías

- **Framework:** Spring Boot 3.x
- **Base de Datos:** PostgreSQL 15 (`payments\_db`) via AWS RDS — ACID para consistencia financiera
- **Gateway Externo:** Stripe API (tarjeta) + Banco Central API (Pix via PSP parceiro)
- **Idempotencia:** Chave de idempotência obrigatória em todos endpoints de criação
- **Webhooks:** Endpoint seguro para receber callbacks do gateway (validação de assinatura HMAC)

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/payments`	Iniciar transacción de pago
`GET`	`/v1/payments/{id}`	Consultar status de pago
`POST`	`/v1/payments/{id}/refund`	Solicitar reembolso
`GET`	`/v1/payments/appointment/{appointmentId}`	Pago vinculado a una consulta
`POST`	`/v1/webhooks/stripe`	Recibir eventos del gateway Stripe

`POST`	`/v1/webhooks/pix`	Recibir confirmaciones de Pix
--------	--------------------	-------------------------------

Dependencias

- **`audit-service` (SQS):** Todo evento de transacción financiera es publicado para auditoría
- **`agendio-scheduling-service` (SQS Consumer):** Recibe `payment-confirmed` para confirmar consulta
- **Stripe API (REST externo):** Gateway de cartão
- **PSP Parceiro (REST externo):** Gateway Pix

Modelo de Datos (payments_db)

...

transactions → id (UUID), appointment_id, patient_id, amount, currency,
status, method, gateway_tx_id, idempotency_key, created_at

refunds → id, transaction_id (FK), amount, reason, status, created_at

payment_methods → id, patient_id, type (PIX/CARD), stripe_token, last4, is_default

receipts → id, transaction_id (FK), pdf_url, issued_at

...

3.7 `medical-records-service` — Historial Clínico (LGPD Compliance)

Repositorio Git: `git@github.com:santopegasus/medical-records-service.git`

Squad Owner: Squad Clínico

Tech Lead: Ana Paula Fonseca

Puerto: 8086

Responsabilidad

El servicio más sensible del ecosistema desde la perspectiva de protección de datos. Gestiona el historial clínico de los pacientes: anamneses, diagnósticos, prescripciones, resultados de exámenes y archivos adjuntos (PDFs, imágenes). El acceso a los datos clínicos es estrictamente controlado por Role-Based Access Control (RBAC) y toda consulta a registros es auditada. El diseño del servicio cumple integralmente con los requisitos de la LGPD: consentimiento explícito del paciente es requerido para compartición de datos, y los registros son anonimizables por solicitud.

Tecnologías

- **Framework:** Spring Boot 3.x + Spring Security (RBAC granular)
- **Base de Datos:** MongoDB via AWS DocumentDB (`medical\_records\_db`) — esquema flexible para evolución de formularios clínicos
- **Almacenamiento de Archivos:** AWS S3 (exámenes em PDF, imagens DICOM) con URLs pre-firmadas de duración limitada (15 minutos)

- **Criptografía:** Campos sensibles (diagnósticos, anotaciones del médico) encriptados en reposo con AWS KMS
- **Anonimización:** Pipeline de anonimización basado en requisición formal, ejecutado como job asíncrono

Reglas de Acceso RBAC

Rol	Permiso
`ROLE_PATIENT`	Leer sus propios registros únicamente
`ROLE_DOCTOR`	Leer y escribir registros de sus pacientes activos
`ROLE_ADMIN_CLINIC`	Leer (sin datos sensibles) registros de su clínica
`ROLE_AUDITOR`	Leer metadatos de acceso (sin contenido clínico)
`ROLE_SYSTEM`	Acceso de sistema para integraciones internas

APIs Expuestas

Método	Endpoint	Descripción
`POST`	`/v1/records`	Crear registro clínico (solo DOCTOR)
`GET`	`/v1/records/patient/{patientId}`	Listar historial del paciente
`GET`	`/v1/records/{id}`	Consultar registro específico
`PUT`	`/v1/records/{id}`	Actualizar registro (solo DOCTOR propietario)
`POST`	`/v1/records/{id}/attachments`	Adjuntar archivo (PDF/imagen)
`GET`	`/v1/records/{id}/attachments/{fileId}`	URL pre-firmada S3 (TTL 15min)
`POST`	`/v1/records/patient/{patientId}/consent`	Registrar consentimiento de compartición
`POST`	`/v1/records/patient/{patientId}/anonymize`	Solicitar anonimización (LGPD Art. 18)

Dependencias

- **`auth-service` (JWT validation):** Extrae roles del token para RBAC
- **`audit-service` (SQS):** Todo acceso a registros clínicos genera evento de auditoría
- **AWS S3:** Almacenamiento de archivos adjuntos
- **AWS KMS:** Gestión de llaves de encriptación en reposo

3.8 `ai-assistant-service` — Asistente de IA con Arquitectura RAG

Repositorio Git: ``git@github.com:santopegasus/ai-assistant-service.git``

Squad Owner: Squad IA

Tech Lead: Fernanda Rocha

Puerto: 8087

Responsabilidad

Provee capacidades de inteligencia artificial a la plataforma Agendio. La funcionalidad principal es un asistente de triaje inteligente que ayuda al paciente a identificar la especialidad médica más adecuada para sus síntomas, basado en una arquitectura RAG (Retrieval-Augmented Generation). El servicio usa LangChain4j para la orquestación del flujo de IA, Pinecone como vector database para el store de conocimiento médico indexado, y un LLM externo (OpenAI GPT-4o) como modelo generativo. Ninguna credencial de la API de IA es hardcodeda en el código; todas las keys son inyectadas via AWS Secrets Manager en tiempo de ejecución.

Tecnologías

- Framework: Spring Boot 3.x + LangChain4j
- LLM: OpenAI GPT-4o (via API externa)
- Vector Store: Pinecone (embeddings de conocimiento médico)
- Embeddings: OpenAI text-embedding-3-large
- Caché de Respuestas: Redis (TTL 1h para consultas frecuentes / idénticas)
- Rate Limiting Interno: Límite de 50 consultas/paciente/día para control de costos

APIs Expuestas

Método	Endpoint	Descripción
`POST`	<code>`/v1/ai/triage`</code>	Triagem de sintomas, sugere especialidade
`POST`	<code>`/v1/ai/chat`</code>	Chat assistido com histórico de contexto
`GET`	<code>`/v1/ai/chat/{sessionId}/history`</code>	Histórico de conversa da sessão
`POST`	<code>`/v1/ai/knowledge/index`</code>	Indexar novo documento médico (Admin only)
`DELETE`	<code>`/v1/ai/chat/{sessionId}`</code>	Encerrar e excluir sessão (LGPD)

Flujo RAG Interno

...

1. Paciente envía pregunta sobre síntomas
2. ai-assistant-service genera embedding de la pregunta
3. Consulta Pinecone para recuperar top-K chunks de documentos médicos relevantes

- 4. Construye prompt enriquecido (pregunta + contexto recuperado)
 - 5. Envía prompt al LLM (GPT-4o)
 - 6. Retorna respuesta + referencias a las fuentes usadas
 - 7. Registra interacción (anonimizada) para mejora futura del modelo
- ...

Dependencias

- ``user-service` (REST):` Verificar perfil del paciente antes de la triagem
 - ``audit-service` (SQS):` Eventos de uso de IA para compliance
 - `OpenAI API (externo):` LLM y embeddings
 - `Pinecone API (externo):` Vector database
 - `Redis:` Caché de respuestas y sesiones de chat
-

3.9 ``audit-service`` — Registro de Auditoría y Compliance

Repositorio Git: ``git@github.com:santopegasus/audit-service.git``

Squad Owner: Squad Governance

Tech Lead: Marco Antônio Lima

Puerto: 8088

Responsabilidad

El ``audit-service`` es el sistema de registro histórico inmutable de todas las acciones sensibles del ecosistema de Santo Pegasus. Es consumidor puro — no expone APIs de escritura a otros servicios; recibe todos sus datos exclusivamente via mensajería SQS. Provee APIs de consulta para auditores internos y para geração de relatórios de compliance. Los registros de auditoría jamás son modificados o eliminados; únicamente son archivados en AWS S3 Glacier tras 90 días para retención de largo plazo.

Tecnologías

- **Framework:** Spring Boot 3.x
- **Base de Datos:** PostgreSQL 15 (``audit_db``) via AWS RDS — immutable append-only
- **Archivado:** AWS S3 Glacier para retención de largo plazo (regulatory compliance)
- **Mensajería:** AWS SQS Consumer exclusivo (múltiples colas)
- **Búsqueda:** Índices PostgreSQL optimizados para queries por Trace ID, user_id, timestamp

APIs Expuestas (Solo Lectura — Roles Restringidos)

Método	Endpoint	Rol Requerido
<code>`GET`</code>	<code>`/v1/audit/events`</code>	<code>`ROLE_AUDITOR`</code> , <code>`ROLE_ADMIN`</code>
<code>`GET`</code>	<code>`/v1/audit/events/{traceId}`</code>	<code>`ROLE_AUDITOR`</code>
<code>`GET`</code>	<code>`/v1/audit/events/user/{userId}`</code>	<code>`ROLE_AUDITOR`</code>

`GET`	`/v1/audit/report/monthly`	`ROLE_ADMIN`
`GET`	`/v1/audit/events/service/{serviceName}`	`ROLE_AUDITOR`

Modelo de Datos (audit_db)

...

audit_events → id (UUID), trace_id, span_id, service_name, action,
actor_id, actor_role, resource_type, resource_id,
payload_summary (NO dados sensíveis), timestamp, ip_address

...

SECCIÓN 4 — MAPA DE DEPENDENCIAS ENTRE SERVICIOS

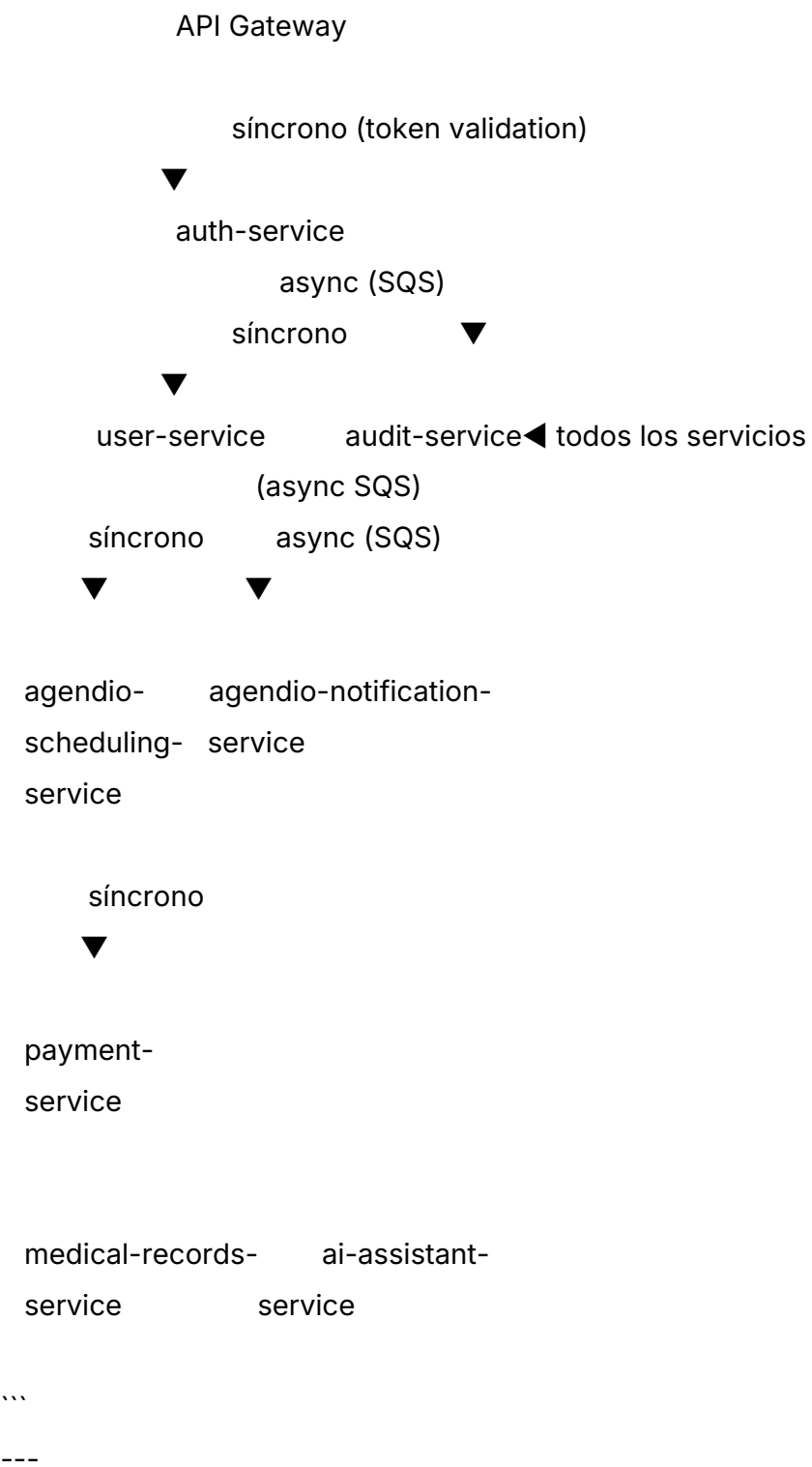
4.1 Tabla de Dependencias

Servicio Consumidor	Servicio Proveedor	Tipo	Protocolo	Criticidad
`api-gateway`	`auth-service`	Síncrono	REST / Redis Cache	Alta
`agendiao-scheduling-service`	`user-service`	Síncrono	REST (WebClient)	Alta
`agendiao-scheduling-service`	`payment-service`	Síncrono	REST (WebClient)	Alta
`payment-service`	`audit-service`	Asíncrono	SQS	Média
`agendiao-scheduling-service`	`agendiao-notification-service`	Asíncrono	SQS	Média
`agendiao-scheduling-service`	`audit-service`	Asíncrono	SQS	Média
`user-service`	`agendiao-notification-service`	Asíncrono	SQS	Baixa
`user-service`	`audit-service`	Asíncrono	SQS	Média
`auth-service`	`user-service`	Síncrono	REST (WebClient)	Alta
`auth-service`	`audit-service`	Asíncrono	SQS	Média
`medical-records-service`	`audit-service`	Asíncrono	SQS	Alta
`ai-assistant-service`	`user-service`	Síncrono	REST	Baixa

`ai-assistant-service`	`audit-service`	Asíncrono	SQS	Média
------------------------	-----------------	-----------	-----	-------

4.2 Diagrama de Grafo de Dependencias

...



SECCIÓN 5 — PATRONES DE COMUNICACIÓN

5.1 REST Síncrono con WebClient

Para comunicaciones síncronas entre servicios, Santo Pegasus adopta el uso del Spring WebClient (no-bloqueante, reactivo), disponible en el módulo `spring-webflux`. La elección de WebClient sobre el deprecated RestTemplate se justifica por su mejor uso de threads y soporte nativo a operaciones non-blocking.

Reglas obligatorias para comunicación REST síncrona:

- Todos los clients HTTP entre servicios utilizan mTLS para autenticación mútua
- Timeout de conexión: 3 segundos (configurable por variable de entorno)
- Timeout de lectura: 10 segundos
- Circuit Breaker via Resilience4j en todas las llamadas a servicios externos
- Retry con backoff exponencial (max 3 tentativas, jitter de 500ms)
- Las respuestas de error deben seguir RFC 7807 (Problem Details)

```
```java
// Ejemplo canónico de WebClient con Resilience4j
@Service
public class UserServiceClient {
 private final WebClient webClient;
 private final CircuitBreaker circuitBreaker;
 public Mono<UserProfileDTO> getUserProfile(UUID userId) {
 return circuitBreaker.executeSupplier(() ->
 webClient.get()
 .uri("/v1/users/{id}", userId)
 .retrieve()
 .onStatus(HttpStatusCode::is4xxClientError, this::handleClientError)
 .bodyToMono(UserProfileDTO.class)
 .timeout(Duration.ofSeconds(10))
);
 }
}
```
```

5.2 gRPC para Alta Performance

Para comunicaciones de alta frecuencia y baja latencia entre servicios internos que no requieren pasar por el API Gateway — como el `agendio-scheduling-service` consultando disponibilidad en tiempo real — Santo Pegasus adopta gRPC como protocolo de comunicação.

Serviços que implementam gRPC (atualmente):

- `agendario-scheduling-service` → `user-service` (endpoint de validação rápida de médico)
- `auth-service` → `user-service` (endpoint de verificação de status em login)

Vantagens adotadas:

- Protocol Buffers (protobuf) garantem payload menor e serialização mais rápida que JSON
- HTTP/2 multiplexing reduz latência em chamadas concorrentes
- Contrato fortemente tipado via `.proto` files versionados em repositório compartilhado

```
```protobuf
```

```
// user-service.proto (exemplo)
```

```
syntax = "proto3";
```

```
service UserService {
```

```
 rpc ValidateUser (ValidateUserRequest) returns (ValidateUserResponse);
```

```
}
```

```
message ValidateUserRequest {
```

```
 string user_id = 1;
```

```
}
```

```
message ValidateUserResponse {
```

```
 bool is_active = 1;
```

```
 string role = 2;
```

```
 string full_name = 3;
```

```
}
```

```
```
```

5.3 Mensageria Assíncrona com AWS SQS

Para comunicação entre domínios (cross-domain events), adotamos AWS SQS FIFO Queues para garantir ordenação e exactly-once delivery. A mensageria assíncrona é o padrão preferido para eventos que não requerem resposta imediata, desacoplando os produtores dos consumidores.

Padrões obrigatórios para SQS:

- Filas FIFO para garantir ordenação e deduplicação (`MessageDeduplicationId` obrigatório)
- Dead Letter Queues (DLQ) configuradas com `maxReceiveCount = 3` para todas as filas principais
- Idempotência no lado consumidor: toda operação deve ser segura para reprocessamento
- Envelope de evento padronizado:

```
```json
```

```
{
```

```
 "eventId": "uuid-v4",
```

```
"eventType": "appointment.created",
"source": "agendio-scheduling-service",
"traceId": "propagado-do-header-http",
"timestamp": "2026-06-16T10:30:00Z",
"schemaVersion": "1.0",
"payload": {
 "appointmentId": "uuid",
 "patientId": "uuid",
 "doctorId": "uuid",
 "scheduledAt": "2026-06-20T14:00:00Z"
}
}
...

```

## SECCIÓN 6 — ESTRATEGIA DE BASES DE DATOS

### 6.1 Database per Service Pattern

Cada microservicio posee y controla exclusivamente su propia base de datos. El acceso directo a la base de datos de otro servicio está terminantemente prohibido. La integración entre datos de distintos dominios ocurre exclusivamente via APIs o mensajería.

### 6.2 Mapa de Bases de Datos por Servicio

Servicio	Motor	Instancia AWS	Base de Datos	Justificativa
`auth-service`	PostgreSQL 15	RDS Multi-AZ	`auth_db`	Consistencia fuerte para credenciales
`user-service`	PostgreSQL 15	RDS Multi-AZ	`users_db`	Dados relacionais de perfil
`agendio-scheduling-service`	PostgreSQL 15	RDS Multi-AZ	`scheduling_db`	ACID para agendamentos críticos
`payment-service`	PostgreSQL 15	RDS Multi-AZ	`payments_db`	ACID para transações financeiras
`medical-records-service`	MongoDB 6	AWS DocumentDB	`medical_records_db`	Esquema flexível para registros clínicos
`ai-assistant-service`	Pinecone	Pinecone Cloud	`knowledge_vectors`	Vector store para RAG

`audit-service`	PostgreSQL 15	RDS Multi-AZ	`audit_db`	Append-only, alta integridade
Caché Global	Redis 7	ElastiCache	Múltiplos namespaces	TTL-based caching cross-service

6.3 Redis — Estrategia de Caché

Namespace	Contenido	TTL	Propietario
`auth:token:{iti}`	Blacklist de tokens revogados	Até expiração do JWT	`auth-service`
`auth:jwtks`	JWKS keys públicas (cache no API GW)	5 minutos	`auth-service`
`user:profile:{userId}`	Perfil do usuário serializado	10 minutos	`user-service`
`schedule:availability:{doctorId}:{date}`	Slots disponíveis do médico	2 minutos	`scheduling-service`
`ai:response:{queryHash}`	Respostas de IA cacheadas	1 hora	`ai-assistant-service`

6.4 Migrations con Flyway

Todas las bases de datos PostgreSQL utilizan Flyway para versionado de schema. Las reglas son:

- Scripts nombrados como `V{versao}\_{descripcion}.sql` (ej: `V2\_\_add\_appointment\_notes\_column.sql`)
- **Scripts de migração são imutáveis após merge em `main`**
- Migrations são executadas automaticamente no startup do serviço (integradas ao pipeline CI/CD)
- **Scripts DDL manuais em produção são terminantemente proibidos**

---

SECCIÓN 7 — API GATEWAY

7.1 Responsabilidades

El API Gateway es el único punto de entrada para todos los clientes externos. Su posición en la arquitectura le otorga responsabilidades críticas de seguridad, observabilidad y gestión de tráfico:

Responsabilidad	Detalles
Routing	Redirige cada ruta para o microserviço correspondente com base no path prefix (`/auth/`, `/users/`, `/appointments/`)

Autenticación Centralizada	Valida token JWT en cada requisição (via Redis cache del `auth-service`) antes de enrutar
Rate Limiting	100 req/min por tenant (ajustável por plano); 10 req/min para endpoints sensibles
SSL/TLS Termination	Toda comunicação externa usa TLS 1.3; internamente se usa mTLS
CORS	Configuração centralizada de origens permitidas por ambiente
Request Logging	Todo request é logado com Trace ID para observabilidade
Circuit Breaking	Se um serviço interno não responde, o Gateway retorna 503 imediatamente
Transformaciones	Adição de headers internos (X-User-Id, X-User-Role, X-Tenant-Id) para context propagation

## 7.2 Tecnología

- **Kong Gateway deployado em AWS ECS Fargate**
- Plugins utilizados: `jwt`, `rate-limiting`, `request-id`, `prometheus`, `cors`, `acl`
- **Configurado via Kong Admin API com `Declarative Config (deck)` versionado em Git**

## 7.3 Mapeo de Rutas

...

/v1/auth/ → auth-service:8081  
/v1/users/ → user-service:8082  
/v1/doctors/ → user-service:8082  
/v1/appointments/ → agendio-scheduling-service:8083  
/v1/schedules/ → agendio-scheduling-service:8083  
/v1/payments/ → payment-service:8085  
/v1/records/ → medical-records-service:8086  
/v1/ai/ → ai-assistant-service:8087  
/v1/audit/ → audit-service:8088 (somente roles AUDITOR/ADMIN)

...

---

# SECCIÓN 8 — INFRAESTRUCTURA EN AWS

## 8.1 Mapa de Servicios AWS Utilizados

Servicio AWS	Uso	Servicio(s) Consumidor(es)
--------------	-----	----------------------------



ECS Fargate	Orquestación de contenedores Docker (sin EC2 gestionado)	Todos los microservicios
RDS PostgreSQL Multi-AZ	Bases de datos relacionales con failover automático	auth, user, scheduling, payment, audit
AWS DocumentDB	MongoDB-compatible para documentos clínicos flexibles	`medical-records-service`
ElastiCache (Redis)	Caché distribuido y blacklist de tokens	auth, user, scheduling, ai-assistant
SQS FIFO	Mensajería asíncrona entre dominios	Todos (producer/consumer)
SES	Envío de emails transaccionales	`agendio-notification-service`
SNS	Envío de SMS	`agendio-notification-service`
S3	Archivos de exámenes, PDFs, backups de audit	medical-records, audit
S3 Glacier	Archivado de logs de auditoría (90+ días)	`audit-service`
Secrets Manager	Gestión de credenciales, llaves de API, certificados	Todos los microservicios
KMS	Encriptación de datos sensibles en reposo	`medical-records-service`
CloudWatch	Logs, métricas nativas AWS, alarmes básicos	Todos
ECR	Registry privado de imagens Docker	CI/CD Pipeline
Route 53	DNS y health checks	API Gateway
Certificate Manager (ACM)	Certificados TLS	API Gateway, Load Balancer
VPC + Private Subnets	Isolamento de red — microservicios nunca expostos publicamente	Todos

## 8.2 Topología de Red

...

AWS VPC (10.0.0.0/16)

Public Subnets

[ALB / API Gateway] [NAT Gateway]

Private Subnets (App)

[ECS Fargate Tasks — Microservicios]

[ElastiCache Redis]

Private Subnets (Data)

[RDS PostgreSQL Multi-AZ]

[DocumentDB Cluster]

...

### 8.3 Estrategia de Deploy — ECS Fargate

- Cada microservicio es una ECS Service dedicada con auto-scaling basado en CPU (target: 60%)
- Task Definitions versionadas; rollback en < 2 minutos via ECS rolling deployment
- Imagens Docker almacenadas en AWS ECR con scanning de vulnerabilidades automático
- Blue-Green deployment via AWS CodeDeploy para servicios críticos (`auth-service`, `agendario-scheduling-service`, `payment-service`)
- Health checks configurados en cada ECS Task apuntando para `/actuator/health`

---

## SECCIÓN 9 — OBSERVABILIDAD DISTRIBUIDA

### 9.1 Los Tres Pilares

La observabilidad en Santo Pegasus se basa en los tres pilares fundamentales descritos en la Guía de Ingeniería Back-end: logs, métricas y rastreo distribuido.

## 9.2 Propagación del Trace ID

El `Trace ID` es el hilo conductor que permite rastrear el ciclo de vida completo de una requisición a través de todos los microservicios involucrados.

### Flujo de propagación:

...

Cliente → API Gateway (genera Trace ID si inexistente)

→ header: X-B3-TraceId: abc123, X-B3-SpanId: span01

→ auth-service (propaga headers, genera Span propio)

→ header: X-B3-TraceId: abc123, X-B3-SpanId: span02

→ agendio-scheduling-service (propaga, genera Span propio)

→ header: X-B3-TraceId: abc123, X-B3-SpanId: span03

→ SQS Message (Trace ID incluido no envelope do evento)

→ agendio-notification-service (propaga via envelope SQS)

→ audit-service (propaga via envelope SQS)

...

### Implementación técnica:

- **Spring Cloud Sleuth (integrado com Micrometer Tracing)** para propagação automática
- **OpenTelemetry Collector** deployado como sidecar em cada ECS Task
- **Dados de tracing** exportados para Datadog APM via OTLP

## 9.3 Logging Estructurado

Conforme a Guía de Ingeniería Back-end (v2.4.0):

- **SLF4J + Logback** como stack de logging em todos os serviços
- **Logs em formato JSON estruturado com campos padronizados:** `traceId`, `spanId`, `service`, `level`, `message`, `timestamp`, `userId` (quando disponível)
- **Nível predeterminado em produção: INFO;** nível DEBUG somente via feature flag temporária
- **Prohibido registrar información sensible (PII):** contraseñas, CPF, datos de pago
- **Logs ingeridos em Datadog Log Management via CloudWatch Logs subscription filter**
- En el proyecto Agendio, el ID da consulta pode ser registrado para rastreio, mas os dados pessoais dos pacientes devem ser omitidos

## 9.4 Métricas con Spring Boot Actuator + Micrometer

- Cada servicio expone `/actuator/metrics`, `/actuator/health`, `/actuator/prometheus`
- **Prometheus coleta métricas via scraping (ECS Service Discovery)**
- **Datadog recebe métricas via integração nativa Prometheus → Datadog Agent**

**Métricas clave monitoreadas por servicio:**

Métrica	Descripción	Alerta configurada
`http.server.requests.duration`	Latência P95 por endpoint	> 2s → Warning; > 5s → Critical
`http.server.requests.error.rate`	Taxa de erros 5xx	> 1% → Warning; > 5% → Critical
`jvm.memory.used`	Uso de memória JVM	> 85% → Warning
`db.pool.connections.active`	Conexiones activas al pool DB	> 80% da pool → Warning
`sqs.messages.received`	Throughput de mensagens SQS	Anomalia detectada → Alert
`circuit.breaker.state`	Estado do Circuit Breaker	OPEN → Critical imediato

9.5 Dashboards en Datadog

Dashboard	Squad	Contenido
Platform Overview	Todos	Visão geral saúde de todos os serviços
Auth & Security	Squad Hermes	Login rates, token errors, anomalias de segurança
Scheduling Core	Squad Agendio Core	Agendamentos/hora, taxa de cancelamento, latência
Payments	Squad Pagamentos	Volume transaccional, taxa de erro de gateway, chargeback
Clinical Records	Squad Clínico	Acessos a registros, erros RBAC, uploads S3
AI Assistant	Squad IA	Latência de inferência, custo estimado de tokens LLM
Governance	Squad Governance	Eventos de auditoria, anomalias de compliance

SECCIÓN 10 — ESTRATEGIA DE VERSIONADO DE APIS

10.1 Política de Versionado

Todas as APIs de Santo Pegasus seguem versionado explícito via path prefix (`/v1/`, `/v2/`). O versionado via headers (`Accept: application/vnd.santopegasus.v2+json`) foi avaliado e descartado por dificultar a observabilidade e o roteamento no API Gateway.

Reglas:

- Novas versiones de API são criadas somente quando há breaking changes

- Adições retrocompatíveis (novos campos opcionais, novos endpoints) não requerem nova versão
- **Uma versão obsoleta (deprecated) permanece ativa por mínimo 6 meses após a publicação da nova versão**

## 10.2 Proceso de Deprecación

...

### FASE 1 — Anuncio (Día 0)

- Publicar changelog no repositório e canal #engineering-announcements
- Adicionar header: Deprecation: true, Sunset: <data> em todas respostas
- Atualizar documentação Swagger com banner de deprecation



### FASE 2 — Período de Suporte (Mês 1 ao 6)

- Ambas versões ativas e suportadas
- Métricas de adoção monitoradas no Datadog
- Comunicação ativa para clientes ainda na versão antiga



### FASE 3 — Sunset (Mês 6+)

- Versão antiga retorna HTTP 410 Gone com link para migração
- Remoção do código após 30 dias de 410

...

## 10.3 Versiones Actuales por Servicio

Servicio	Versión Activa	Versión Deprecated	Sunset
`auth-service`	v1	—	—

`user-service`	v1	—	—
`agendiao-scheduling-service`	v2	v1	Dez/2026
`payment-service`	v1	—	—
`medical-records-service`	v1	—	—
`ai-assistant-service`	v1 (beta)	—	—
`audit-service`	v1	—	—

## SECCIÓN 11 — SEGURIDAD ENTRE SERVICIOS

### 11.1 Autenticación y Autorización

Conforme documentado na Guia de Ingeniería Back-end (v2.4.0), o controle de acceso es gestionado de forma centralizada utilizando Spring Security. Todos os endpoints são protegidos; nenhum endpoint interno é acessível sem validação de identidade. Tokens JWT com fluxo OAuth 2.0 / OpenID Connect são o mecanismo padrão. A extração e validação de permissões (roles) ocorrem a nível de endpoint.

### 11.2 mTLS para Comunicación Interna

Toda comunicação serviço-a-serviço dentro da VPC privada usa mTLS (Mutual TLS). Isso garante que ambos os lados de uma conexão se autenticuem, prevenindo ataques de spoofing e man-in-the-middle em comunicações internas.

Implementación:

- **Certificados gerenciados por AWS Certificate Manager (ACM) Private CA**
- Certificados rotacionados automaticamente a cada 90 días
- Cada microserviço possui um certificado de cliente único para autenticação mútua
- **Service Mesh (AWS App Mesh) como camada de controle de mTLS (em rollout progressivo — ver Roadmap)**

### 11.3 Service Accounts y Principio de Privilegio Mínimo

Cada microservicio opera com um IAM Role exclusivo na AWS, seguindo estritamente o principio de privilegio mínimo:

Servicio	IAM Role	Permisos concedidos
`auth-service`	`iam-role-auth-service`	Secrets Manager (auth/), ElastiCache, SQS (publish audit-events)

`agendiao-scheduling-service`	`iam-role-scheduling-service`	RDS (scheduling_db), SQS (publish/consume scheduling queues)
`medical-records-service`	`iam-role-medical-records`	DocumentDB, S3 (medical-files/), KMS, SQS (publish audit-events)
`payment-service`	`iam-role-payment-service`	RDS (payments_db), Secrets Manager (payment/), SQS
`ai-assistant-service`	`iam-role-ai-assistant`	Secrets Manager (ai/), ElastiCache, SQS
`audit-service`	`iam-role-audit-service`	RDS (audit_db), SQS (consume all audit queues), S3 Glacier

**Credenciais de servicios externos (Stripe, OpenAI, Pinecone): armazenadas exclusivamente em AWS Secrets Manager e injetadas em tempo de execução via Spring Cloud Config. Jamais escritas no código ou repositórios Git. Ferramentas de escaneio automático (`gitleaks`, `trufflehog`) são executadas nos pipelines de CI/CD para detectar e bloquear commits que contenham secrets.**

## 11.4